

Temporal Evidence Maintenance for Revisable Monitoring Explanations

Anonymous Authors

Anonymous Institution

Abstract. Monitoring explanations can remain visible after their supporting evidence changes. We evaluate explicit temporal dependencies for revising generated numerical explanations, separating direct checking, transitive maintenance, and storage implementation. A prospectively frozen study uses one synthetic benchmark, WESAD, and PPG-DaLiA: 60 held-out episodes, four replay variants, three checkpoints, and five methods produce 3,600 initial requests and 1,620 bounded repairs. In common symbolic fixtures, full graph and indexed relational maintenance correct all four required claim changes; direct-only maintenance corrects two. In the waveform task, however, all checked methods have zero observed display violations under the strict query contract and correct every eligible old claim; transitive propagation adds no measured benefit. Contract violations include accurately stated off-target background facts, so these rates are not hallucination estimates. Full-graph required-fact recall is 76.7–79.3%, and useful coverage is 55.5–60.5%. A separate open-loop workload tests 1, 5, and 20 events/s. The graph misses 21 of 40 one-second flag targets and 40 of 40 five-second adequate-replacement targets. None of the 236 systems answers is complete. The results distinguish evidence withdrawal from useful regeneration and quantify implementation cost without treating graph storage as an intrinsic source of factuality.

Keywords: Temporal knowledge graphs · Evidence provenance · Generative explanations · Incremental maintenance · Physiological streams

1 Introduction

A monitoring explanation can become unreliable after it has been generated. A delayed packet can overturn a statement that a measurement is unavailable; a corrected observation can invalidate a numerical comparison; and a quality notice can remove one of several sources supporting a statement. Generating a fresh answer does not, by itself, repair an earlier answer that remains on a screen. The system must identify which displayed statements depend on the changed evidence and decide whether each still has sufficient support.

Consider a display containing three statements: the target electrodermal activity (EDA) median is 2 units, its difference from an earlier median of 1 unit is 1 unit, and an independent source also supports a value of 2 units. If the target is corrected to 4 units, the first statement and the dependent comparison

require attention. The independently supported statement can remain. A blanket deletion of all reachable statements loses useful content, while updating only direct citations can leave an incorrect downstream comparison visible. The distinction between direct checking, transitive maintenance, and alternative support is the object of this study.

We investigate whether an explicit temporal evidence representation improves the reliability and maintenance of generated monitoring explanations while preserving useful answers. We separately ask whether graph storage offers an operational advantage over indexed relational dependency tracking. These are different questions: a storage engine can change update cost without changing the semantics of support. A useful evaluation must therefore include both a direct-checking ablation and a relational implementation with equivalent maintenance capabilities.

Our experimental package uses one synthetic benchmark in two modes—exact symbolic maintenance fixtures and waveform replay—plus recordings from WESAD and PPG-DaLiA. The real recordings test evidence handling under two acquisition conditions, not diagnosis or physiological causation. We evaluate 60 held-out base episodes, four replay variants, three knowledge-time checkpoints, and five methods. A separate open-loop workload measures update queues and the latency of generated replacements. All generative and neural verification computation uses a CUDA GPU; runtime checks and independent exact evaluation operate on recorded numerical evidence.

The contribution is a controlled, auditable evaluation of evidence maintenance for generated explanations. It combines immutable source versions, a bounded claim contract, direct and transitive checks, persistent explanation versions, and a strong relational control. The package retains raw model responses, unsuccessful development runs, intervention manifests, paired case identifiers, and scripts producing the tables and figures. We assess errors jointly with answer coverage, distinguish retraction from useful replacement, and separate deterministic maintenance tests from outcomes that depend on generated claims. We do not propose a new general truth-maintenance algorithm or treat an external dependency graph as a reconstruction of a language model’s internal reasoning.

2 Related Work

Dependency-directed belief revision has a long history. Doyle’s truth-maintenance system records reasons for beliefs and revises their status when assumptions change [2]. Incremental view maintenance studies how database query results can be updated after changes to their inputs, including alternative derivations [5]. Provenance semirings offer an algebraic account of contributing and alternative derivations in relational queries [4]. Our finite acyclic AND/OR predicates use these established ideas in a bounded explanation task. They do not replace general database provenance semantics. This lineage also motivates the relational control: dependency maintenance is not exclusive to property graph databases.

Table 1. Positioning by evaluated task. B2 in this study is a custom propagation ablation, not a reproduction of GraphRAG or Graphiti.

Work	Evidence and updates	Evaluation emphasis
Truth/view maintenance	Dependencies and alternative derivations	Correct revision of beliefs or query results
GraphRAG	Entity/community summaries of text	Global summarization quality
Zep/Graphiti	Temporal conversational facts and invalidation	Long-term agent memory
HEG-TKG	Clinical literature and tempo-ral anchors	Citation traceability and clinical review
Internal reasoning graphs	Model attribution dependencies	Hallucination detection
This study	Versioned numerical evidence and displayed claims	Correction, retained information, and storage cost

GraphRAG constructs entity and community representations to support global question-focused summarization of text corpora [3]. Its evaluated objective concerns answer comprehensiveness and diversity. Our endpoint concerns the continued validity of previously displayed numerical statements after evidence changes. Zep’s Graphiti engine organizes conversational memory with temporal facts, source episodes, and contradiction-based invalidation; its evaluation includes Deep Memory Retrieval and LongMemEval [9]. Temporal graphs and fact invalidation are consequently prior capabilities, rather than novelty claims of this paper.

The HEG-TKG preprint grounds rare-disease reasoning in traceable clinical literature and temporal disease-stage information, and evaluates citation verification, clinician judgments, and injected-evidence counterfactuals [1]. It is a close example of evidence-linked temporal reasoning. Our experiment instead uses numerical sensor replay, exact dependency fixtures, paired coverage metrics, and a graph-versus-relational systems comparison. It neither reproduces that clinical system nor evaluates clinical decision quality.

An evidence graph also has a limited relationship to interpretability. Turpin et al. show that chain-of-thought explanations can omit input factors that influence model answers [14]. Hu et al. construct semantic internal reasoning graphs from model attribution for hallucination detection [6]. Our edges encode external evidence requirements imposed by a task contract. Intervening on them tests traceability and maintenance behavior; it does not establish that the model used those edges internally. Table 1 summarizes these distinctions without claiming that every capability of the related implementations was reproduced.

3 Temporal Evidence Maintenance

3.1 Evidence, Knowledge Time, and Query Scope

A record r has an immutable identifier, a logical identity, a version number, an event interval $[s_r, e_r]$, and an ingestion time τ_r . The event interval describes the observation; ingestion time describes when the system learned it. The snapshot available at knowledge time t is

$$E_t = \{r : \tau_r \leq t\}. \quad (1)$$

For each logical identity, current queries select the greatest visible version, breaking ties by ingestion time and identifier. A historical query reconstructs the snapshot at its own knowledge time. An older version is not retrospectively false merely because a newer version subsequently arrived.

A query specifies dataset, subject, session, target interval, knowledge time, and required quantities. Matching a current target interval is distinct from being a recent record in storage. In this experiment, a direct numerical claim must describe the requested 30-second interval. A comparison additionally requires the immediately preceding 120-second interval. The same baseline value can therefore be valid as a comparison operand and invalid as an unqualified answer to the target-window question. This explicit scope avoids using a single arbitrary expiry threshold for all facts.

3.2 Representation and Storage

The schema contains Subject, Sensor, Observation, FeatureVersion, ClaimVersion, ExplanationVersion, and ReviewEvent records. Raw arrays remain outside the database. Observation records contain source hashes, native sample ranges, sampling rates, and channel identifiers. Feature records point to their observation windows and record extractor identity and units. A generated candidate is retained as a proposal; successful parsing or a model-assigned confidence does not promote it to verified evidence.

DEPENDS_ON points from a dependent item to a required item, so reverse traversal identifies potentially affected descendants. SUPERSEDES points from a new version to the previous version. DERIVED_FROM connects features to observations; CONTAINS connects explanations to claims; SUPPORTS connects accepted sources to claims. Subject, sensor, contradiction, and review links complete the schema. Support assessments are append-only records indexed by item, knowledge time, and triggering revision.

The primary implementation uses Neo4j Community 5.26.0 with a compound identity constraint and indexes for scope, dataset, subject, session, event interval, ingestion time, logical identity, and version. Batch writes use UNWIND, and reverse dependency traversal uses variable-length paths. The relational control uses SQLite in WAL mode, indexed evidence and edge tables, and recursive common table expressions. Both implementations cache immutable record objects

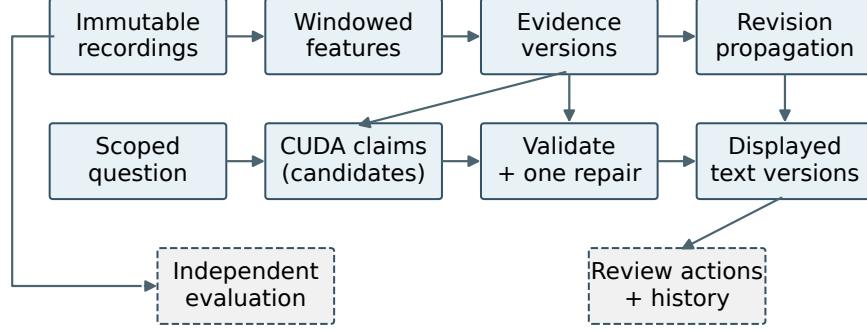


Fig. 1. Evidence and explanation lifecycle. Completed recording windows produce versioned features. A common bounded generator proposes claims; checks determine displayed content. Later events trigger dependency maintenance and immutable explanation revisions. Independent evaluation reads source/replay logs. Dashed boundaries separate evaluator-only labels and isolated review actions.

in the application process and issue database queries for snapshots and affected identifiers. They share support predicates and batch assessment writes. This is a comparison of these implementations on one host, including Neo4j’s service boundary, not a universal comparison of graph and relational engines.

3.3 Bounded Generation and Direct Validation

All conditions use the same Qwen3-8B model revision [8], query text, serialized admissible evidence, sampling settings, and JSON schema. The contract permits at most three claims and six claim types: numerical observation, trend, comparison, evidence conflict, missing evidence, and revision effect. Each claim includes a quantity, value, unit, subject, interval, operator, evidence identifiers, parent-claim identifiers, and one sentence. The explanation must concatenate the claim sentences. The answer also declares whether it is answered, partially answered, or lacks sufficient evidence. Structured decoding constrains syntax, not correctness [15].

Direct checks verify reference existence, dataset, subject and session identity, query-relative version and interval, units, numerical agreement, required comparison operands, and parent-claim support. Numerical agreement uses

$$|v - \hat{v}| \leq \max(10^{-6}, 0.01|\hat{v}|), \quad (2)$$

where $\hat{v}$ is the recorded reference value. This is a reporting tolerance, not a clinical decision threshold. Cyclic proposed claim dependencies are rejected. Explicitly invalidated evidence cannot serve as current support. Missingness claims are

checked against currently available evidence rather than against an oracle fault label.

The runtime additionally checks whether the sentence contains the stated value and unit, flags a limited set of unsupported causal or clinical terms, and checks that the paragraph matches its claims. These lexical checks do not guarantee semantic fidelity. We retain the original prose and perform a separate automated fidelity audit. Checked conditions receive at most one repair generation with the same evidence and the failed checks. After that attempt, unsupported claims are removed and the paragraph is recomposed from retained sentences. Empty outputs remain in evaluation and reduce coverage; they do not receive a perfect claim-accuracy score.

3.4 Revision Propagation and Persistent Explanations

For a revision, the system appends the new record, preserves the old citation, and traverses reverse dependencies of every earlier version of the same logical item. Considering all earlier versions matters for a sequence $v_1 \rightarrow v_2 \rightarrow v_3$: a displayed claim may still cite v_1 when v_3 arrives. The affected subgraph is evaluated in dependency order. For an AND expression, every required operand must remain supported. For an OR expression, one adequate alternative suffices. Derived numeric expressions are recomputed before dependent claims are assessed. Reachability therefore identifies candidates for revision, not a set that must all be deleted.

The core algorithm has the following steps:

1. Append the evidence version and its supersession link.
2. Retrieve affected dependents, with direct or transitive reach according to the condition, at the event’s ingestion time.
3. Recompute their support predicates in acyclic dependency order and append the resulting assessments.
4. Withdraw unsupported displayed sentences, preserve supported alternatives, and persist a new explanation version when its text changes.

When support returns, the original immutable sentence can be restored in a later explanation version. Negative-evidence claims subscribe to availability records so a delayed packet can invalidate a prior statement of absence. Old explanation text remains accessible for historical inspection.

Immediate maintenance is conservative text retraction or recomposition. A useful new numerical answer requires separate generation. In the matched replay, fresh generation occurs at the three scheduled checkpoints; it is not an asynchronous regeneration of every historic paragraph. The systems workload separately requests a GPU replacement after a material adverse transition. We report withdrawal and generated-replacement latency separately. This prevents a fast status update from being described as a fast complete answer.

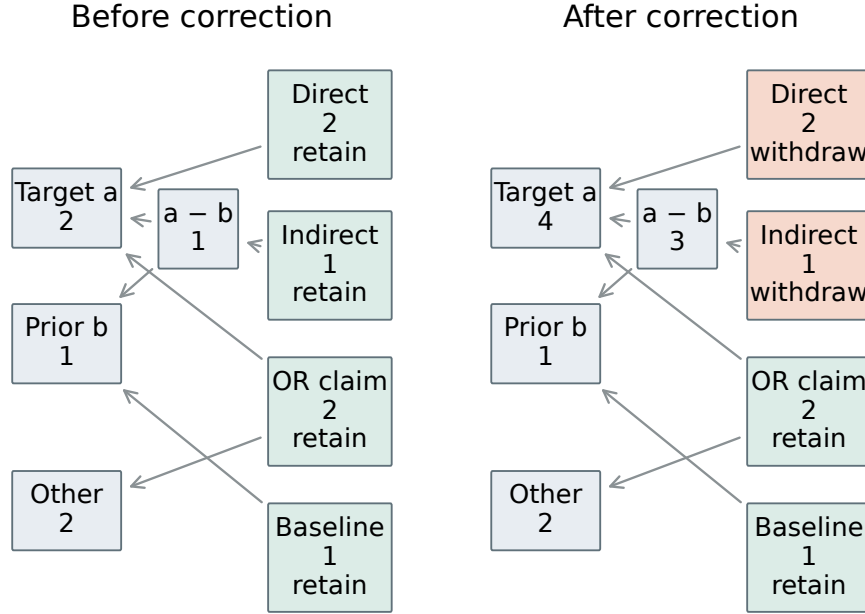


Fig. 2. Exact fixture before and after a correction. A target changes from 2 to 4; its difference from a baseline of 1 changes from 1 to 3. The direct and downstream old claims become contradicted. The OR-supported claim remains supported by an independent value of 2, and the baseline claim is unaffected. Arrows point from a dependent to its required source. This is a constructed correctness example, not a measured physiological event.

4 Experimental Design

4.1 Three Sources and Participant-Level Partitions

The synthetic benchmark generates 30-minute sessions with pulse-like cardiac signals, respiratory oscillations, slow EDA components with response-like events, and three-axis motion. Subject offsets and noise vary; motion and the latent physiological-style process use separate random streams. Sampling rates are 128, 32, 8, and 32 Hz for cardiac, respiratory, EDA, and motion channels, respectively. These are generator choices, not a validated physiological simulator. Seeds 101–110 define development subjects and 1001–1010 define held-out subjects. Separate seeded streams determine waveform generation, fault assignment, and question assignment. Latent values and intervention labels remain outside runtime prompts and validation.

WESAD provides laboratory recordings from 15 participants with chest and wrist signals [13,12]. PPG-DaLiA provides recordings from 15 participants during daily activities [11,10]. We inventory the original synchronized files and retain their acquisition URLs, hashes, documentation, and stated usage terms.

Table 2. Data and evaluation inventory. A case is one method at one checkpoint. Raw scalar values count feature-processing inputs, including overlapping windows and replay extraction; they are not independent observations.

Source	Dev/test subjects	Test episodes	Scenarios	Cases	Raw values
Synthetic	10/10	20	80	1,200	1,900,800
WESAD	5/10	20	80	1,200	6,220,800
PPG-DaLiA	5/10	20	80	1,200	6,220,800

Neither laboratory protocol labels nor activity annotations enter the generator. PPG-DaLiA reference heart-rate targets and R peaks are also withheld. The recordings supply evidence values; replay revisions do not imply that the original investigators published erroneous measurements.

For each real source, naturally sorted subject identifiers are permuted with NumPy PCG64 seed 20260928. Five subjects form the development partition and ten form the held-out partition. Every episode, variant, and checkpoint from a subject remains in that partition. The synthetic benchmark similarly has ten development and ten test subjects. We select two target windows at 25% and 75% of each session’s common channel duration, rounding down to a five-second grid. Eligibility requires at least 600 seconds of synchronized coverage; all acquired sessions qualify. The selected contexts do not overlap within a session. Selection uses recording duration, not model performance or hidden condition labels. Table 2 is generated from the saved inventories and held-out episode manifests.

4.2 Windowed Features and Controlled Replays

Features use native rates and fully observed completed windows. A 30-second target is compared with the immediately preceding 120-second interval. Seven overlapping update windows end at five-second increments leading to the target. We compute EDA median and interquartile range, missing fraction, a repeated-extrema quality cue, and acceleration-magnitude standard deviation. Wrist EDA uses microsiemens. Synchronized WESAD acceleration uses recorded $1/64g$ units, whereas PPG-DaLiA’s synchronized acceleration uses g ; a development alignment check against 300 original acceleration samples verified the latter scaling.

Optional spectral summaries of cardiac and respiratory signals use a Hann-windowed periodogram restricted to declared bands. A rate is withheld when data are insufficient, nonfinite, or lack the required spectral concentration of 0.5. These summaries are available context, not independently validated physiological estimates; the required answer slots rely on directly verifiable EDA, missingness, and motion features. Motion alone does not establish sensor failure. Numerical extraction uses NumPy on the host, with a fixed-sample CUDA quantile cross-check; there is no learned feature model or neural CPU fallback.

Every base episode has four paired variants:

1. *Clean*: same-value target EDA revisions at matched times.

Table 3. Five matched conditions. “Direct” means revisiting direct references when their evidence changes. Full maintenance includes transitive dependencies and alternative-support evaluation.

ID	Storage	Checks	Maintenance	Repair allowance
B0	Indexed records	No	No	No
B1	Indexed records	Yes	Direct	One
B2	Neo4j	Yes	Direct	One
M1	Neo4j	Yes	Full	One
B3	SQLite + recursion	Yes	Full	One

2. *Channel fault*: either the last 15 seconds of EDA become missing or an offset of 0.5 is added to the window. Altered arrays are separate hashed files; derived feature versions cite their actual replay observation versions.
3. *Delayed*: the target EDA feature packet arrives 60 seconds later, while retaining its original event interval.
4. *Correction*: the target median receives an explicit value revision of +0.5, followed by restoration of the original value.

Checkpoints are at 1, 31, and 91 seconds after the target window ends. Fault and correction revisions appear at the latter two checkpoints; the delayed packet arrives at offset 61 seconds. The clean variant follows the same checkpoint schedule. Channel faults are split evenly into missingness and corruption within each source. Independent seeded rankings balance question families and fault types, with both fault types present in each source/family group. A hidden offset is not assumed detectable before an explicit correction: faithful reporting of the currently ingested value is not scored as omniscient recognition of corruption.

4.3 Questions, Conditions, and Call Budget

Each source has five episodes in each of four question families. Current-evidence and conflict/timing questions require the target EDA median and its signed difference from the prior median. Sufficiency questions require the target median and missing fraction. Revision-impact questions ask about the prior display after an update; their scored numerical slots are the current median and target motion variability. Fact recall does not separately score completeness of the revision narrative; persistent-display scores measure changes to earlier statements. Development and test wordings differ while retaining these fixed answer slots. All methods receive the same current evidence serialization. For revision-impact questions, the preceding B0 display is common context across methods; it is an observed candidate answer, not a supplied gold answer.

Table 3 separates fresh generation from maintenance of earlier displays. B0 receives structured recent evidence but no post-generation semantic checks and does not update old answers. B1 uses checked flat evidence and direct reference maintenance. B2 applies those same checks over graph storage with transitive

propagation disabled. M1 and B3 enable full maintenance in graph and relational storage, respectively. All checked methods receive the same one-repair allowance. Their fresh generations need not be identical because a fixed seed does not remove hardware and batching effects; logical correctness is therefore also tested with a common fixed candidate fixture.

The held-out matrix comprises $60 \times 4 \times 3 \times 5 = 3,600$ initial calls. At most 2,880 validation repair calls are possible, for a hard ceiling of 6,480. Four independent scenarios share a four-request semaphore; each scenario must finish its current checkpoint before exposing its next one. Scenario order is a fixed seeded permutation. Manifests, episode hashes, prompts, configuration, runtime source hashes, and model revisions are frozen before held-out generation. Interrupted requests with unknown outcomes are recorded rather than silently regenerated. Resumption uses immutable case identifiers and request journals.

The final configuration uses Qwen3-8B in BF16, non-thinking mode, temperature 0.7, top- p 0.8, top- k 20, and seed 20260928. The input allowance is 4,096 tokens and the common output ceiling is 1,536 tokens. Development pilots exposed truncation and constrained-decoder whitespace loops at smaller budgets; their failures are retained. The final prompt explicitly names every required field, and xgrammar emits compact JSON. No model weights are trained or tuned for this study. The final pilot comprises 100 development cases and a separate 20-request concurrency-one latency workload. Hardware and measured costs are reported with the results.

4.4 Independent Outcomes and Uncertainty

The evaluator reconstructs visible versions and required answer slots directly from immutable replay logs. It does not import runtime validation or use stored support status as gold truth. It checks claims, evidence references, query scope, numbers, units, dependencies, and limited sentence/paragraph consistency. A second, separately pinned CUDA model audits 60 nonempty final explanations, four per method/source, sampled by a fixed permutation at checkpoint 2. Four additional diagnostic examples test the verifier. We retain prompts, outputs, parse failures, and disagreements with exact checks. This is automated annotation using Qwen2.5-3B-Instruct [7]; common model-family errors remain possible.

For fresh answers, case error is the fraction with any contract-invalid displayed claim or paragraph discrepancy. Claim error divides contract-invalid claims by emitted checkable claims. Required-fact recall divides correctly supplied slots by slots answerable from the current snapshot. Useful coverage requires all currently answerable slots in an eligible query. We report abstentions, partial answers, parse/truncation failures, timeouts, and repair calls. Claim error is undefined for an empty answer, and a failed or empty case remains in the fixed case denominator and coverage metrics. Faithfulness to ingested evidence is explicitly separate from physiological estimation accuracy.

For persistent displays, a claim requires change if it was valid when created and becomes invalid under the current snapshot. Correction completeness counts those claims withdrawn or revised appropriately; residual incorrect display counts

those still active. Collateral revision counts withdrawal of content that remains valid. Initially incorrect claims are recorded separately. An absent required-change denominator yields an undefined correction rate, not a perfect score. Replay service times exclude offered-load queues; the systems track measures arrival-to-completion latency separately.

Two prospective principal contrasts are M1 versus B2 for correction completeness, and M1 versus B1 for case error jointly with required-fact recall. M1 versus B3 is a separate implementation comparison. Paired observations stay grouped by subject, preserving all episode variants and checkpoints. First, each metric is computed within a base episode across its variants and checkpoints; paired episode differences are averaged within each subject. We use 2,000 paired subject-cluster bootstrap resamples and pointwise 95% intervals, with ten independent held-out subjects per source. Conditional correction contrasts include only episodes with defined denominators in both methods and subjects with at least one such pair. We disclose both retained counts and give subjects equal weight. Pooled descriptive rates use their displayed denominators and need not equal the paired estimand. We report effect sizes without hypothesis-test p -values; the intervals do not provide simultaneous family-wise coverage. Waveform samples, duplicated events, and repeated prompts do not increase the participant count.

4.5 Open-Loop Systems Workload

The systems track gives M1 and B3 the same immutable symbolic fixture, including an AND-dependent comparison and surviving OR support. A producer thread schedules arrivals independently of completions at 1, 5, and 20 derived events per second, for 60 seconds per cell. Method order alternates across rates. The target value switches between 2 and 4 every 20 events after an initial ten-event offset. Each implementation performs its own database retrieval and updates. A scheduled explanation is requested every 20 events, plus a replacement when a formerly supported claim becomes contradicted. Four requests may generate concurrently; queues are allowed to grow and then drain.

The separate ceiling is 156 scheduled explanations, 80 affected-explanation requests, and at most 236 validation repairs. We record arrival-to-flag time, arrival-to-generated-replacement time, database and retrieval time, pending explanations, event backlog, throughput including drain, and tokens. Flagging within one second and an adequate replacement within five seconds are measured soft targets. Replacement adequacy is initially checked against the request snapshot; completion-time event indices permit a separate stale-answer check. Logical assessment hashes must agree between databases for the common sequence. These repeated symbolic events measure system load, not additional physiological subjects or raw waveform throughput.

Table 4. Fresh displayed answers by source. All rates are percentages. Recall counts answerable required slots; coverage requires all such slots. An empty answer contributes no checkable claims and reduces coverage. Errors are violations of the bounded query contract, including interval scope.

Source	Method	Case err.%	Claim err.%	Recall%	Coverage%	Claims
Synthetic	B0	52.9	43.2	78.8	60.0	588
Synthetic	B1	0.0	0.0	79.3	60.5	335
Synthetic	B2	0.0	0.0	79.3	60.5	336
Synthetic	M1	0.0	0.0	79.3	60.5	336
Synthetic	B3	0.0	0.0	79.3	60.5	335
WESAD	B0	59.2	39.2	76.0	54.1	710
WESAD	B1	0.0	0.0	76.9	55.9	440
WESAD	B2	0.0	0.0	76.9	55.9	440
WESAD	M1	0.0	0.0	76.9	55.9	440
WESAD	B3	0.0	0.0	76.7	55.5	439
PPG-DaLiA	B0	56.2	38.5	76.9	55.9	693
PPG-DaLiA	B1	0.0	0.0	76.9	55.9	425
PPG-DaLiA	B2	0.0	0.0	77.1	56.4	427
PPG-DaLiA	M1	0.0	0.0	76.7	55.5	425
PPG-DaLiA	B3	0.0	0.0	76.9	55.9	426

5 Results

5.1 Accounting and Fresh-Answer Reliability

The locked run accounts for all 3,600 planned initial cases, with 1,620 bounded repairs and 5,220 total request attempts. There are 3 failed final cases and 3 initial request/format failures, including 0 truncated outputs. The journals record 0 client timeout attempts across initial and repair phases. The matched experiment takes 2.89 hours on the recorded hardware. Failed, empty, and partially answered cases remain in their fixed denominators. Table 4 reports case error together with claim error, required-fact recall, and useful coverage for each source and method. Each source/method cell contains 240 query checkpoints. The three failed requests are transport ReadErrors in B0, one per source; they were retained without retries.

All checked methods have zero observed case and claim violations under the exact contract. B0 case violation rates are 52.9–59.2%. M1 retains 76.7–79.3% of answerable required facts, but supplies all answerable slots for only 55.5–60.5% of eligible queries. The checked controls have closely similar recall and coverage.

The distinction between contract compliance and factual correctness materially affects these results. A post-hoc descriptive check finds that 333 of B0’s 799 invalid claims are direct numerical background statements that pass the same exact checks at their own stated interval. For example, a correctly labelled preceding-window EDA median violates the target-interval rule. A comparison declaring that statement as a parent can consequently fail the dependency check despite

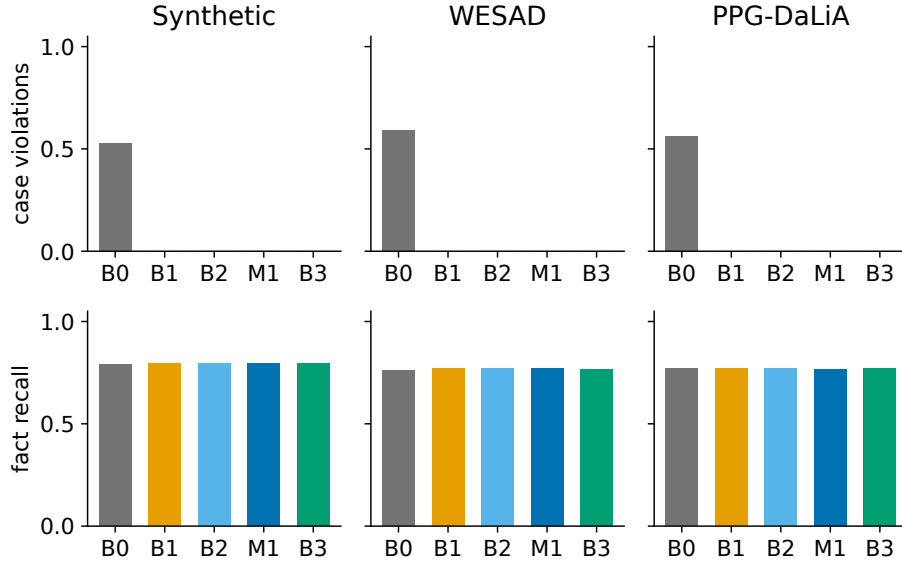


Fig. 3. Query-contract case violations (top) and required-fact recall (bottom) for the three waveform sources. The five bars within each panel use the same evidence and generation settings. Lower error is desirable only alongside retained recall. These are aggregate proportions; paired subject-cluster intervals are reported for the prospective contrasts in Table 7.

correct arithmetic. Table 12 gives source-wise counts. The frozen primary scores are unchanged; their error reductions must not be read as equivalent reductions in hallucinations, and their recall also depends on this contract.

These comparisons describe the final displayed content after the common bounded repair policy. A low error rate cannot alone establish useful answering: removing an unsupported sentence can lower error while leaving a required value absent. Figure 3 places error and recall on the same source-wise comparison. Table 11 exposes abstention, reference validity, repairs, and failures. Error-component counts and answerability-stratified partial answer/abstention counts are available in the reproducible analysis tables.

5.2 Required Changes and Retained Content

The exact symbolic fixtures isolate maintenance from generation. They contain two claims requiring change after a correction or invalidation, an independently supported OR claim, and an unaffected baseline claim. Benign and irrelevant updates test stability. Table 5 aggregates these four constructed interventions. Full graph and full relational maintenance identify both direct and downstream required changes and retain alternative support. Direct-only maintenance identifies the direct change while leaving the downstream old claim active. Restoration

Table 5. Common symbolic fixture. Required changes occur in correction and invalidation modes; benign and irrelevant modes contribute unaffected claims. The identical candidate structure is replayed into every method.

Method	Corrected/required	Residual	Collateral/unaffected
B0	0/4	4	0/12
B1	2/4	2	0/12
B2	2/4	2	0/12
M1	4/4	0	0/12
B3	4/4	0	0/12

is also checked by replaying a third version. These are deterministic correctness cases, not independent participant observations, and no sampling interval is attached to them.

Generated persistent displays have a different denominator: the method must first produce a valid claim that later requires change. Table 6 therefore reports the actual corrected/required and collateral/unaffected counts, as well as residual incorrect active content. Figure 4 displays the corresponding proportions. These are claim-checkpoint opportunities, and their number can differ across methods because initial answers differ. Initially incorrect content is tracked separately rather than credited as correctable content that the method once supported validly.

All four checked methods correct every eligible generated claim and have no collateral withdrawals in this run. Thus, the waveform task supplies no measured correction advantage for transitive propagation over direct checking. B0 retains every eligible outdated claim. The symbolic and generated results answer different questions and should not be pooled.

M1 emits 1201 displayed claims, of which 18 declare parent claims. The bounded numerical contract also requires direct evidence references. This structure constrains the opportunity for transitive maintenance to add value over direct checking in the waveform task. The common fixture demonstrates a deliberately necessary transitive path; its result should not be substituted for an empirical advantage across unrestricted generated reasoning chains. No generated persistent claim has a declared parent dependency as its sole later exact-check error in this run.

5.3 Paired Contrasts

Table 7 reports the two prospective principal comparisons by source. Differences are M1 minus the control in percentage points, computed within subjects. Negative case-error differences favor M1; positive recall and correction differences favor M1. The table keeps these outcomes separate so error reductions cannot hide lost information. Conditional correction contrasts disclose their eligible subject and episode counts. Pointwise bootstrap intervals quantify participant-level variability within this fixed generation run; they do not measure variability across model

Table 6. Persistent generated displays. Completeness is conditional on an initially valid claim requiring change. Residual counts those still active; collateral counts withdrawal of content still valid. Undefined rates are shown as dashes.

Source	Method	Corrected/ required	Complete (%)	Residual	Collateral/ unaffected
Synthetic	B0	0/71	0.0	71	0/245
Synthetic	B1	73/73	100.0	0	0/247
Synthetic	B2	73/73	100.0	0	0/247
Synthetic	M1	73/73	100.0	0	0/247
Synthetic	B3	73/73	100.0	0	0/247
WESAD	B0	0/71	0.0	71	0/336
WESAD	B1	74/74	100.0	0	0/349
WESAD	B2	74/74	100.0	0	0/349
WESAD	M1	74/74	100.0	0	0/349
WESAD	B3	73/73	100.0	0	0/348
PPG-DaLiA	B0	0/80	0.0	80	0/326
PPG-DaLiA	B1	80/80	100.0	0	0/329
PPG-DaLiA	B2	80/80	100.0	0	0/330
PPG-DaLiA	M1	80/80	100.0	0	0/328
PPG-DaLiA	B3	80/80	100.0	0	0/330

seeds or remove shared measurement error. The zero differences and zero-width intervals for the principal correction and case-error contrasts arise because all observed paired subject differences are identical. They do not establish universal equivalence or a zero error probability.

All checked conditions use the same fresh-answer checks and repair policy on matched evidence. Small M1–B1 differences therefore do not isolate an additional fresh-answer mechanism; persistent displays supply the maintenance comparison. The M1–B3 differences in Table 13 compare generated outcomes under equivalent support semantics. Fresh outputs can still differ under the same requested seed because scheduling and batching are not bitwise identical. The deterministic fixtures and systems assessment hashes supply the separate test of logical equivalence. Differences in generated answers must not be attributed to graph storage alone.

5.4 Open-Loop Latency and Resource Use

All six systems cells complete, and graph/relational logical assessment hashes agree for every offered rate. Table 8 reports flag latency, time until candidate completion, deadline misses, and peak queues from arrival. Figure 5 displays the load dependence. The M1 implementation misses 21 of 40 flag targets and 40 of 40 adequate-replacement targets. Replacement failure includes inadequate content as well as lateness. Fast withdrawal therefore does not imply a useful replacement within five seconds. All 236 systems displays contain a single target observation and omit the required difference, even after the bounded repair

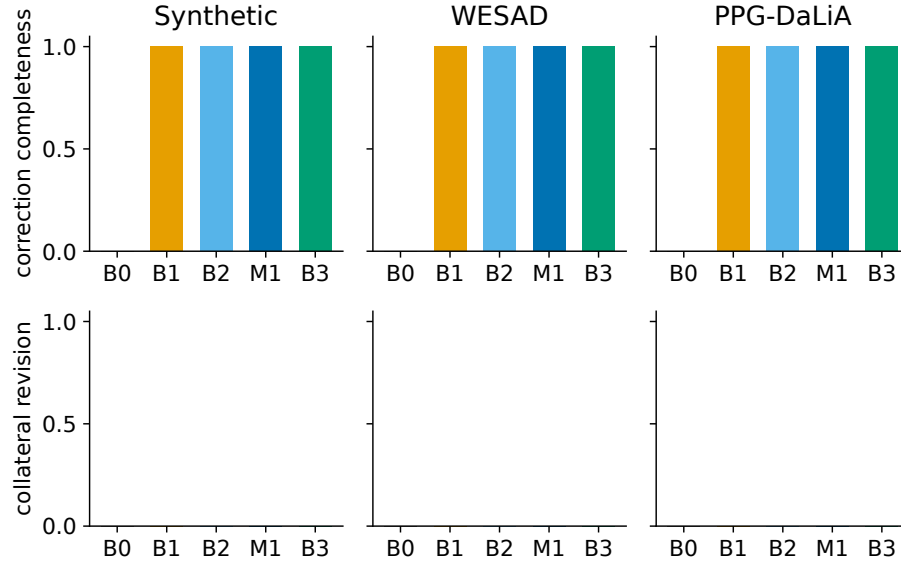


Fig. 4. Persistent-display correction completeness (top) and collateral withdrawal (bottom). Denominators appear in Table 6. A zero collateral proportion has a different interpretation from an undefined correction rate; undefined bars are labeled n/a. These are replay outcomes, not open-loop arrival-to-replacement measurements.

attempt. Successful-replacement latency is therefore undefined; the reported candidate latency measures when these incomplete attempts finish.

At one and five events/s, both backends meet every flag deadline, and the relational implementation has lower p95 flag latency. At 20 events/s both accumulate event and answer queues and miss flag deadlines. M1 has lower p95 flag and candidate latency at that rate, but more flag misses. This reversal across summaries cautions against a single storage-engine ranking.

The frozen adequacy check uses the request snapshot. A descriptive check also remaps evidence to the latest ingested versions at completion (Table 14), preserving the original scores. With no adequate input-snapshot answers, zero answers “became stale”; this is uninformative about preserving useful replacements. Rechecking before publication and coalescing obsolete requests remain unevaluated scheduling policies.

The experiment uses one RTX 5090; Table 10 gives hardware and model details. The development workload with one concurrent request has median and p95 generation latencies of 13.34 and 17.07 seconds, including bounded repairs. The separate batched pilot completes 0.323 initial cases per second, including repair and database cost. These are different workloads; batch throughput does not establish low interactive latency. The primary worker verifies all parameters and first-forward tensors on CUDA, with no CPU weight offload. The systems track makes 472 requests, within its separate 472-request ceiling. Table 9 includes

Table 7. Principal paired differences with 2,000 subject-cluster resamples. The M1–B1 reliability contrast is interpreted jointly for error and recall. N (S/E) gives retained subjects/episodes. Intervals are pointwise 95%; no hypothesis-test p -values are reported.

Source	Contrast	Outcome	N (S/E)	Difference (pp)	95% interval (pp)
Synthetic	M1–B2	Correction	10/20	0.00	[0.00, 0.00]
Synthetic	M1–B1	Case error	10/20	0.00	[0.00, 0.00]
Synthetic	M1–B1	Fact recall	10/20	0.00	[0.00, 0.00]
WESAD	M1–B2	Correction	10/20	0.00	[0.00, 0.00]
WESAD	M1–B1	Case error	10/20	0.00	[0.00, 0.00]
WESAD	M1–B1	Fact recall	10/20	0.00	[0.00, 0.00]
PPG-DaLiA	M1–B2	Correction	10/20	0.00	[0.00, 0.00]
PPG-DaLiA	M1–B1	Case error	10/20	0.00	[0.00, 0.00]
PPG-DaLiA	M1–B1	Fact recall	10/20	-0.25	[-0.75, 0.00]

Table 8. Open-loop systems results. Each cell offers events for 60 seconds and then drains. Q columns are peak queued events and pending explanations. Flag and repair columns show misses/opportunities for one- and five-second targets. Candidate latency includes inadequate outputs; no complete replacement is observed.

Events/s	Method	Flag p95 (ms)	Candidate p95 (s)	Flag misses	Repair misses	Event Q	Answer Q
1	M1	43.9	11.0	0/2	2/2	0	2
1	B3	3.7	10.9	0/2	2/2	0	2
5	M1	52.8	18.7	0/8	8/8	0	7
5	B3	23.7	19.5	0/8	8/8	0	7
20	M1	33,687.0	218.7	21/30	30/30	347	62
20	B3	77,698.4	237.9	17/30	30/30	394	50

retained development failures, final experiments, and the verifier. Stage wall times include host work and idle gaps and are not CUDA kernel times. One-second telemetry measures observed VRAM and utilization; cgroup memory includes file cache and other processes in the pod. Database node, edge, chain-length and physical-size accounting is included in the artifact report. Physical Neo4j storage also contains development scopes, so its size is not presented as an equal-content compression comparison. No hourly account price was available; we report time and tokens without an invented monetary cost.

5.5 Automated Fidelity Audit and Review Demonstration

The separate verifier parses 60 of 60 sampled nonempty final explanations, flags 4, and disagrees with the exact claim checks on 14. It gives the expected result on 4 of four diagnostic examples. These counts describe an auxiliary automated audit, not a validated estimate of semantic accuracy. Selection is stratified across methods and sources and conditional on a nonempty checkpoint-2 display. Empty outputs are already represented in coverage and abstention metrics. Raw audit prompts

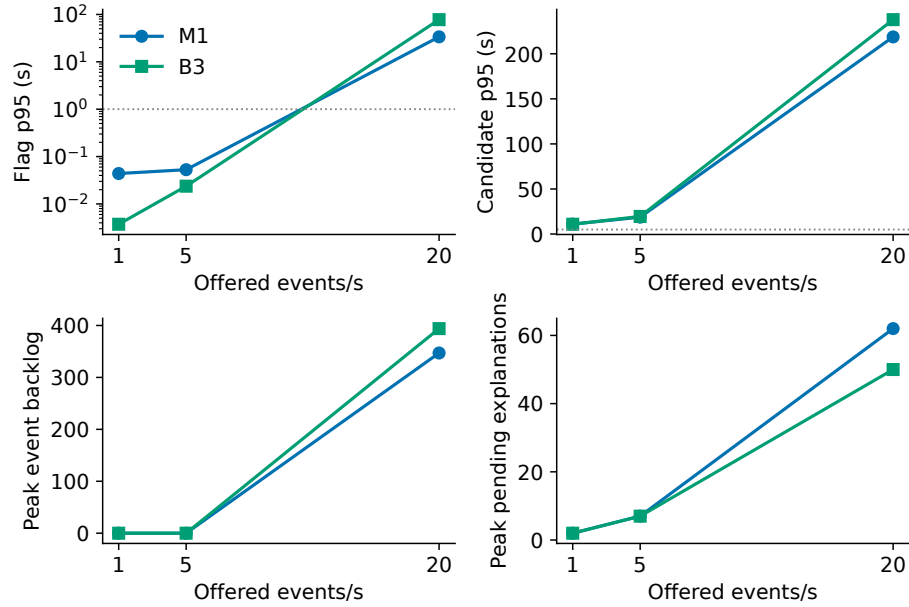


Fig. 5. Offered load versus p95 flag and candidate-completion latency and peak queues for M1 and B3. Dotted lines mark the one-second flag and five-second replacement targets. The flag axis is logarithmic. The independent producer continues scheduling events while requests wait or generate; measurements include the resulting backlog.

and responses permit inspection of both flags and apparent false alarms. The verifier accepts 10 exact-invalid outputs, including incorrect differences and absent citations as well as interval-contract violations. It flags 4 exact-valid outputs; its rationales include treating a null measurement as present and rejecting rounding within the stated tolerance. These visible failures limit its value as a semantic check despite passing the four simple diagnostics.

The local review dashboard demonstrates the maintenance operations in an isolated workspace. An issue queue prioritizes explicit evidence problems and downstream claim count; a selected claim exposes its local graph, source feature values, time windows, and versions. Reviewers can reject evidence, accept a proposed numerical correction, or mark an interpretation unresolved. Actions append ReviewEvents and explanation versions and show before/after text. Scripted examples demonstrate dependent-statement withdrawal, retained OR support, and a mistaken rejection that removes valid content. These examples establish functionality and an observable limitation of review actions; no human performance or review-speed outcome was measured.

Table 9. Measured request and token accounting. Pilot rows include their separate interactive workload. Systems pilot v1 includes requests from its failed scoring smoke test. “No usage” counts attempts without returned token counts: their server-side cost is unknown, not zero. Token sums include available usage only. Wall time excludes package/model downloads and manuscript work.

Workload	Requests	No usage	Input tokens	Output tokens	Wall min
Matched test	5,220	3	8,525,358	2,880,024	173.2
pilot v1	207	0	256,281	103,092	11.4
pilot v2	164	0	230,554	90,664	9.6
pilot v3	186	0	300,657	106,302	9.3
pilot v4	187	0	302,546	106,911	9.3
Systems test	472	0	416,220	206,856	15.1
Automated audit	64	0	143,213	2,786	0.6
Systems pilot v1	8	0	6,792	3,060	0.4
Systems pilot v2	8	0	6,792	3,060	0.4

6 Discussion and Limitations

Direct checking suffices for the measured generated-answer corrections in this bounded task. Full graph propagation matches the simpler controls on contract errors and correction completeness, while many answers remain incomplete. The symbolic fixture demonstrates necessary transitive maintenance; the waveform answers usually cite operands directly, leaving little opportunity for added benefit. A comparison can be withdrawn through its own citation even when it also lists a parent. Future studies should prospectively vary dependency depth and independent support while keeping candidates and answerable facts matched.

Generated parent links are declared prerequisites, not independently proven necessary derivations. A redundant link can cause withdrawal even when direct source checks pass. The waveform evaluation measures maintenance of accepted declarations, not discovery of semantically necessary reasoning graphs. The common fixture supplies the stronger test under identical, known dependencies.

The indexed relational control uses the same support predicates and agrees on logical outcomes. Latency includes an in-process SQLite engine versus a local Neo4j service, a small dependency structure, growing revision chains, and shared host resources. It cannot establish a general ranking for distributed graphs, concurrent writers, richer queries, or separately optimized deployments. One 60-second cell per method and rate supports descriptive measurements only.

The independent evaluator implements the same strict query contract as the runtime. Its rejection of correctly labelled baseline facts restricts both error and recall interpretation. A future protocol should distinguish useful background evidence from misleading time attribution. Recorded-value agreement establishes source faithfulness, not physiological truth. Lexical checks can miss unsupported interpretations or reject equivalent paraphrases. The auxiliary verifier shares the generator’s model family, examines only 60 nonempty outputs, and makes visible

errors despite passing four simple diagnostics. There is no human annotation gold standard or evidence of comprehensive semantic fidelity.

Controlled faults at one severity do not estimate natural fault prevalence. Runtime has no evaluator-only corruption label. The waveform matrix fixes target intervals while knowledge changes; historical scope and out-of-order arrivals are correctness fixtures, not a broad evaluation of moving current-state queries. Optional spectral summaries have no physiological accuracy validation. Ten held-out subjects per source, two episodes per subject, related wearable hardware, one generator, and one seed limit generalization. Subject resampling respects dependence but cannot overcome those limits; conditional correction also depends on what each method originally generated.

The dashboard shows consequences of review actions without certifying a supplied correction. A mistaken rejection can remove valid content. Ranking policies, review accuracy and speed require separate studies. Likewise, retaining useful prior text, coalescing stale requests, and checking replacements immediately before publication remain prospective engineering work. Here, prompt withdrawal and complete regeneration are distinct outcomes, and the latter fails in the systems workload.

7 Conclusion

The three-source evaluation finds no correction benefit from transitive propagation over direct checking in generated waveform answers. All checked methods satisfy the exact display contract, with limited useful coverage and no general factuality guarantee. Full graph and relational maintenance agree on known symbolic dependencies. Flagging succeeds at the two lower offered rates, but the systems workload yields no complete generated replacements. The contribution is an auditable benchmark and bounded maintenance implementation; the results do not establish an intrinsic reliability advantage for graph storage.

Artifacts and reproducibility. The package includes the prospective plan, acquisition and split manifests, frozen hashes, raw requests and failures, independent scores, analyses, resource traces, figure scripts, and an isolated review demonstration. Provider recordings are obtained under their recorded terms and are not redistributed. Setup commands and a report map results to saved inputs.

Acknowledgements and use of generative AI. An AI coding assistant assisted implementation, analysis scripting, literature organization, and drafting. Qwen3-8B generated experimental answers; Qwen2.5-3B-Instruct performed the automated audit. Neither system supplies human annotation or authorship. Human authors retain responsibility for checking the evidence, interpretations and references and approving any submission.

References

1. Ahmed, M.S., Dusanic, M., Kirschner, M.N., Nyoungui, E., Zschüntzsch, J., Poech, L.G., Röttger, R.: The provenance gap in clinical AI: Evidence-traceable temporal knowledge graphs for rare disease reasoning (2026), <https://arxiv.org/abs/2604.17114v2>, preprint, version 2
2. Doyle, J.: A truth maintenance system. *Artificial Intelligence* **12**(3), 231–272 (1979). [https://doi.org/10.1016/0004-3702\(79\)90008-0](https://doi.org/10.1016/0004-3702(79)90008-0)
3. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph RAG approach to query-focused summarization (2025), <https://arxiv.org/abs/2404.16130v2>, version 2
4. Green, T.J., Karvounarakis, G., Tannen, V.: Provenance semirings. In: *Proceedings of ACM PODS*. pp. 31–40 (2007). <https://doi.org/10.1145/1265530.1265535>
5. Gupta, A., Mumick, I.S., Subrahmanian, V.S.: Maintaining views incrementally. In: *Proceedings of ACM SIGMOD*. pp. 157–166 (1993). <https://doi.org/10.1145/170035.170066>
6. Hu, J., Li, Y., Zhong, J., Zou, L., Qi, W.: Detecting hallucinations in retrieval-augmented generation via semantic-level internal reasoning graph. In: *Findings of the Association for Computational Linguistics: ACL 2026*. pp. 27826–27841. Association for Computational Linguistics, San Diego, California, United States (2026). <https://doi.org/10.18653/v1/2026.findings-acl.1385>, <https://aclanthology.org/2026.findings-acl.1385/>
7. Qwen Team: Qwen2.5-3B-Instruct model card (2024), <https://huggingface.co/Qwen/Qwen2.5-3B-Instruct>
8. Qwen Team: Qwen3-8B model card (2025), <https://huggingface.co/Qwen/Qwen3-8B>, revision b968826d9c46dd6066d109eabc6255188de91218
9. Rasmussen, P., Paliychuk, P., Beauvais, T., Ryan, J., Chalef, D.: Zep: A temporal knowledge graph architecture for agent memory (2025), <https://arxiv.org/abs/2501.13956>
10. Reiss, A., Indlekofer, I., Schmidt, P.: PPG-DaLiA. UCI Machine Learning Repository (2019). <https://doi.org/10.24432/C53890>
11. Reiss, A., Indlekofer, I., Schmidt, P., Van Laerhoven, K.: Deep PPG: Large-scale heart rate estimation with convolutional neural networks. *Sensors* **19**(14), 3079 (2019). <https://doi.org/10.3390/s19143079>
12. Schmidt, P., Reiss, A.: WESAD (wearable stress and affect detection). UCI Machine Learning Repository (2018). <https://doi.org/10.24432/C57K5T>
13. Schmidt, P., Reiss, A., Duerichen, R., Marberger, C., Van Laerhoven, K.: Introducing WESAD, a multimodal dataset for wearable stress and affect detection. In: *Proceedings of the International Conference on Multimodal Interaction*. pp. 400–408 (2018). <https://doi.org/10.1145/3242969.3242985>
14. Turpin, M., Michael, J., Perez, E., Bowman, S.R.: Language models don’t always say what they think: Unfaithful explanations in chain-of-thought prompting. In: *Advances in Neural Information Processing Systems* (2023), <https://arxiv.org/abs/2305.04388>
15. vLLM Contributors: Structured outputs (2025), https://docs.vllm.ai/en/v0.10.2/features/structured_outputs.html, version 0.10.2

A Additional Descriptive Results

Table 10. Recorded hardware and model settings. GPU and host peaks are observed over the matched held-out stage; one-second sampling can miss short peaks.

Component	Recorded setting
GPU	RTX 5090; 31.36 GiB
Primary model	Qwen3-8B; BF16; CUDA only
Model revision	b968826d9c46
Inference	vLLM 0.10.2; PyTorch 2.8.0+cu128
Database	Neo4j 5.26.0; SQLite 3.45.1 WAL
Input / output limit	4,096 / 1,536 tokens
Batch concurrency / interactive	4 / 1
CPU quota / memory limit	27.2 CPU equivalents / 94.0 GB
Test GPU memory peak	28,441.0 MiB
Test GPU utilization mean	88.7%
Test host cgroup memory peak	28.5 GiB

Table 11. Generation behavior and supported-reference rates. Failed and truncated columns are counts out of 240 cases per row; repairs count additional requests. Reference validity requires support under the strict query contract.

Source	Method	Failed	Truncated	Repairs	Abstain%	Valid refs%
Synthetic	B0	1	0	0	0.4	48.9
Synthetic	B1	0	0	127	8.3	100.0
Synthetic	B2	0	0	127	8.3	100.0
Synthetic	M1	0	0	127	8.3	100.0
Synthetic	B3	0	0	127	8.3	100.0
WESAD	B0	1	0	0	0.4	52.2
WESAD	B1	0	0	141	8.3	100.0
WESAD	B2	0	0	140	8.3	100.0
WESAD	M1	0	0	141	8.3	100.0
WESAD	B3	0	0	142	8.3	100.0
PPG-DaLiA	B0	1	0	0	0.4	52.9
PPG-DaLiA	B1	0	0	137	8.3	100.0
PPG-DaLiA	B2	0	0	136	8.3	100.0
PPG-DaLiA	M1	0	0	138	8.3	100.0
PPG-DaLiA	B3	0	0	137	8.3	100.0

Table 12. Post-hoc contract-scope breakdown. The last column counts direct numerical observations with only interval-related errors that pass the existing exact checks at their own stated interval and original knowledge time. These source-faithful background facts remain errors in the unchanged primary metric. This narrow breakdown does not exhaust the consequences of interval scope, including failures propagated to dependent comparisons.

Source	B0 contract-invalid claims	Locally supported off-target facts
Synthetic	254	107
WESAD	278	115
PPG-DaLiA	267	111

Table 13. Separate graph-versus-relational comparison of generated outcomes. Differences are M1–B3 in percentage points with pointwise 95% paired intervals; N (S/E) gives retained subjects/episodes.

Source	Outcome	N (S/E)	M1–B3 (pp)	95% interval (pp)
Synthetic	Case error	10/20	0.00	[0.00, 0.00]
Synthetic	Fact recall	10/20	0.00	[0.00, 0.00]
Synthetic	Correction	10/20	0.00	[0.00, 0.00]
WESAD	Case error	10/20	0.00	[0.00, 0.00]
WESAD	Fact recall	10/20	0.25	[0.00, 0.75]
WESAD	Correction	10/20	0.00	[0.00, 0.00]
PPG-DaLiA	Case error	10/20	0.00	[0.00, 0.00]
PPG-DaLiA	Fact recall	10/20	-0.25	[-0.75, 0.00]
PPG-DaLiA	Correction	10/20	0.00	[0.00, 0.00]

Table 14. Descriptive completion-time adequacy. Each explanation is checked against its request snapshot and the latest ingested event when it completes. “Became stale” counts request-adequate answers that no longer satisfy the two required numerical slots. These counts do not replace the frozen metrics.

Events/s	Method	Calls	Input adequate	Completion adequate	Became stale
1	M1	5	0	0	0
1	B3	5	0	0	0
5	B3	23	0	0	0
5	M1	23	0	0	0
20	M1	90	0	0	0
20	B3	90	0	0	0